

Please expand the documentation in the following code.

```
def merge_dicts(base, override):  
    """  
    Merge two dictionaries, with values from  
    override taking precedence.  
    Returns a new dictionary; neither input is  
    mutated.  
    """  
    result = base.copy()  
    for key, value in override.items():  
        if isinstance(value, dict) and key in  
            result and isinstance(result[key], dict):  
            result[key] =  
merge_dicts(result[key], value)  
        else:  
            result[key] = value  
    return result
```

Here's an expanded docstring that explains the behavior, parameters, return value, recursion rules, and provides examples:

<> Python



▶ Run

```
def merge_dicts(base, override):  
    """  
    Recursively merge two dictionaries and return the merged result.  
  
    This function creates a new dictionary by combining the contents of  
    ``base`` and ``override``. Values from ``override`` take precedence over  
    values from ``base`` when the same key exists in both dictionaries.  
  
    If a key exists in both dictionaries and the corresponding values are  
    themselves dictionaries, those nested dictionaries are merged  
    recursively. For all other value types, including lists, tuples, sets,  
    and scalar values, the value from ``override`` completely replaces the  
    value from ``base``.  
  
    Neither input dictionary is modified. A new dictionary object is returned.  
  
    Parameters  
    -----
```

`base : dict`

The default or original dictionary whose values provide the starting state for the merge.

`override : dict`

The dictionary containing values that should override or extend the contents of ``base``.

Returns

`dict`

A new dictionary containing the merged contents of ``base`` and ``override``.

Notes

- Nested dictionaries are merged recursively.
- Non-dictionary values are replaced rather than merged.
- The function performs a shallow copy of dictionaries at each level before applying recursive merges.
- Objects stored as values are not deep-copied unless they are merged through recursive dictionary processing.

Examples

Basic override:

```
>>> merge_dicts(  
...     {"host": "localhost", "port": 8080},  
...     {"port": 9090}  
... )  
{'host': 'localhost', 'port': 9090}
```

Recursive merge:

```
>>> merge_dicts(  
...     {"database": {"host": "db1", "port": 5432}},  
...     {"database": {"port": 5433}}  
... )  
{'database': {'host': 'db1', 'port': 5433}}
```

Adding new keys:

```
>>> merge_dicts(  
...     {"debug": False},  
...     {"timeout": 30}  
... )  
{'debug': False, 'timeout': 30}
```

Replacing non-dictionary values:

```
>>> merge_dicts(  
...
```

```
...     {"items": [1, 2, 3]},
...     {"items": [4]}
... )
{'items': [4]}
"""

result = base.copy()
for key, value in override.items():
    if isinstance(value, dict) and key in result and isinstance(result[key], dict):
        result[key] = merge_dicts(result[key], value)
    else:
        result[key] = value
return result
```

This style is suitable for internal libraries and API documentation generators such as Sphinx, NumPy-style doc tooling, and IDE documentation popups.

